

The background of the slide features a large, light blue watermark of the University of Freiburg seal. The seal is circular and contains a central figure, likely a seated person, surrounded by various heraldic symbols and text in a circular border.

Wie wir arbeiten

Stündliche Läufe am Formalisierungs-Inventar

Peter Pfaffelhuber

Freiburg, September 2026

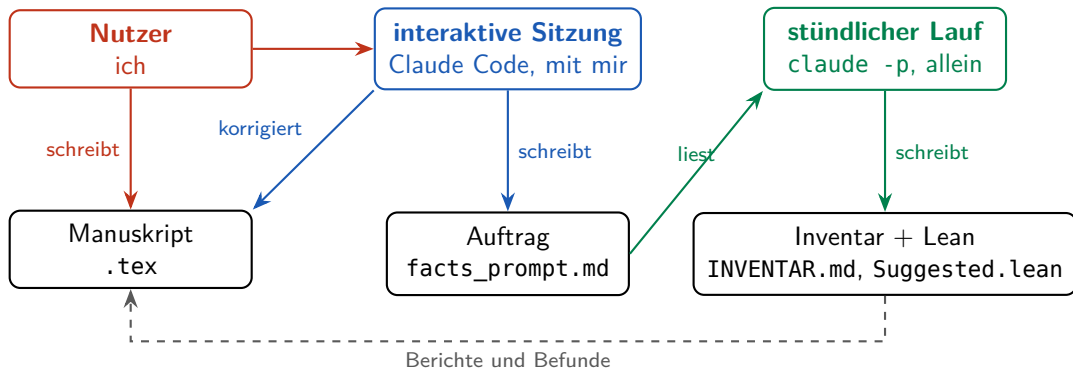
Worum es geht

- ▶ Ein Manuskript über Martingalprobleme, `Journal/Blog/MartingaleProblem/MartingaleProblem.tex`, rund 140 Seiten.
- ▶ Dazu eine **Formalisierung in Lean**, verteilt auf vier Roadmaps: `WeakConvergence`, `SkorokhodSpace`, `MartingaleProblems`, `JumpProcesses`, jeweils mit einer Datei `Suggested.lean`.
- ▶ Die Formalisierung schreiben zum größten Teil **unbeaufsichtigte Läufe** von Claude, **einmal pro Stunde**.
- ▶ Ich schreibe die Aufträge, lese die Berichte und entscheide.

Stand am 26. 9. 2026

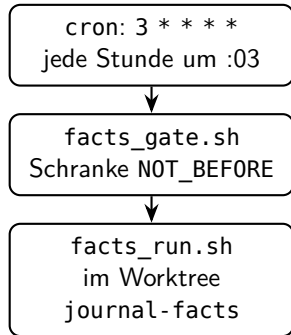
Läufe seit dem 29. 8. 2026	gut 400
Lean-Zeilen in den vier <code>Suggested.lean</code>	rund 110 000
<code>check_master.py</code> am Ende jedes Laufs	0 Fehler, 0 sorry
Axiome jedes neuen Hauptsatzes	<code>propext</code> , <code>Classical.choice</code> , <code>Quot.sound</code>

Drei Beteiligte, vier Dateien



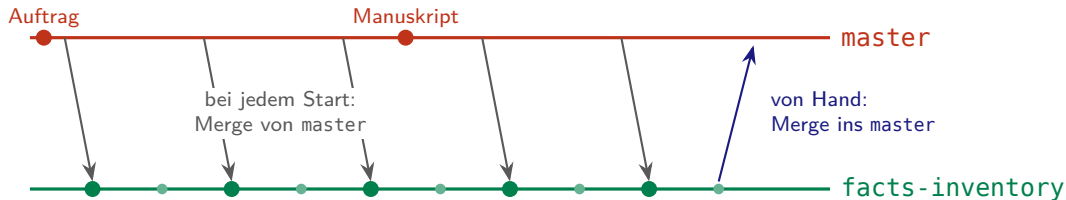
Die README-Dateien der Roadmaps redigiere ich selbst. Kein Lauf fasst sie an.

Der Takt: was jede Stunde passiert



1. **Sperre** (flock): läuft der vorige Lauf noch, fällt dieser Slot aus.
2. **Kontingent**: steht eine Sitzungsgrenze fest, wird bis dahin nicht gestartet.
3. **master nachziehen**: git merge origin/master in den Branch facts-inventory.
4. Commit „Facts STATUS: Lauf ... gestartet“.
5. claude -p mit Opus 5.5, Zeitschranke **120 Minuten**, enge **Werkzeugliste**, Auftrag über stdin.
6. **Schutznetz**: Übersetzt das Manuskript nicht mehr, werden seine Änderungen verworfen.
7. Commit und Push „Facts ... (rc=0)“, auch im Fehlerfall.

Zwei Branches, ein Rückweg



- ▶ Auf master liegen **Manuskript und Auftrag**. Dort schreiben nur ich und die interaktive Sitzung.
- ▶ Auf facts-inventory liegen **Inventar und Lean**. Dort schreiben nur die Läufe, in einem eigenen Worktree. Mein Checkout bleibt unberührt.
- ▶ Ein neuer Auftrag wirkt also erst, wenn er **gepusht** ist, und zwar ab dem nächsten Start.
- ▶ Zurück in master kommt die Arbeit nur durch meinen Merge (181 bisher).

Woher ein Lauf seinen Auftrag bekommt

scripts/facts_prompt.md, bei jedem Start frisch gelesen:

1. **Vorrangige Aufgaben**, von mir datiert, mit **Schritten in fester Reihenfolge**.
2. **Stehende Regeln**: minimale Voraussetzungen, Übersetzen, der Pflichtbericht am Ende.
3. Erledigte Aufgaben wandern nach facts_prompt_archiv.md. Kein Lauf liest das Archiv.

Ein Lauf hat kein Gedächtnis

Jeder Lauf beginnt leer. Was er wissen muss, steht an zwei Stellen:

- ▶ in Facts/INVENTAR.md: Berichte, Zuordnungen „Manuskript $\mapsto$ Lean-Name“, Befunde;
- ▶ in den Commits auf facts-inventory.

Ein Schritt ist ein Lauf und ein benanntes Ziel

Jeder Lauf nennt am Ende das Ziel für den nächsten, und das ist immer der nächste offene Schritt *meiner* Liste.

Was ein Lauf darf und was nicht

Darf

- ▶ Lean schreiben und übersetzen (lake, lean)
- ▶ `check_master.py`: baut alle vier Dateien gegen Mathlib master. Diese Prüfung zählt.
- ▶ Mathlib und BrownianMotion **lesen**
- ▶ Befunde ins Inventar schreiben, mit Zeilennummer

Darf nicht

- ▶ das Manuskript ändern
- ▶ eine README anfassen
- ▶ **nach außen** gehen: kein PR, kein Issue, kein Kommentar
- ▶ Prozesse im Hintergrund starten
- ▶ Dateien löschen (rm ist nicht freigegeben)

Die Rechte hängen am **Namen des Befehls**. Nicht Freigegebenes wird im Modus -p verweigert, nicht erfragt. Ein Lauf bleibt also nie hängen und wartet auf mich.

Mein Einfluss

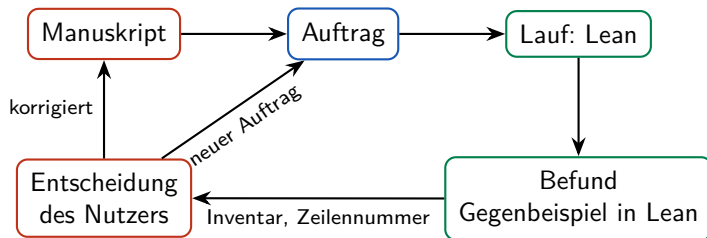
1. **Aufträge stellen.** In einer interaktiven Sitzung formuliert und nach master gepusht. Ich setze die Reihenfolge; Läufe wählen sich kein eigenes Ziel.
2. **Anhalten und freigeben.** Mit NOT_BEFORE in facts_gate.sh, zum Beispiel am 15. 9. (Kontingent erschöpft) und am 22. 9. (keine Läufe mehr, ohne Enddatum), wieder frei am 24. 9.
3. **Das Manuskript korrigieren.** Die Läufe finden Fehler, ich entscheide und ändere.
4. **Entscheiden, was nach außen geht.** Mathlib-Kandidaten und Lücken bei BrownianMotion sammeln sich im Inventar, ich entscheide über sie.
5. **Abnehmen.** Der Merge von facts-inventory nach master, zuletzt am 22. 9.
6. **Aufräumen und nachsteuern,** wenn Läufe festhängen oder sich irren.

Ein Tag: der 26. 9. 2026

-
- 08:32 Neuer Auftrag: A (Martingale in stetiger Zeit), B (lokales Martingalproblem ohne Einschränkungen), C (abstraktes Martingalproblem)
- 09:03 Optional sampling in stetiger Zeit, ohne Schranke an die Pfade und bei roher Filtration
- 10:03 càdlàg-Modifikation, beide Richtungen, mit einem Zeugen, dass Rechtsstetigkeit der Filtration nötig ist
- 13:03 **Fehler im Manuskript:** starke Markoveigenschaft im inhomogenen Fall, Zeuge in Lean
- 16:03 **Lücke:** Chapman–Kolmogorov braucht eine Verträglichkeit der Schiftsysteme
- 17:03 **Lücke:** der verschobene Pfad muss adaptiert sein
- 19:03 Der Lauf korrigiert den Lauf davor: zweimal „offen“, was schon dastand
- 19:30 In der interaktiven Sitzung: Manuskript korrigiert, neuer Auftrag D und E
- 21:03 Der erste Lauf mit dem neuen Auftrag
-

B2 (Konvexität bei roher Filtration) hing fünf Läufe lang fest: Jeder nannte sie als Ziel, keiner hatte einen neuen Gedanken.

Die Rückkopplung



- ▶ Ein Befund ist erst dann ein Befund, wenn er ein **Zeuge in Lean** ist, mit allen Voraussetzungen geprüft. Eine bloße Vermutung reicht nicht.
- ▶ Das Manuskript folgt der Formalisierung, nicht umgekehrt. Geändert wird es aber nur durch mich.

Was schiefging, und was das Skript daraus gelernt hat

Datum	Panne	jetzt
5. 9.	„lake: Befehl nicht gefunden“, weil cron einen leeren PATH hat	PATH im Skript gesetzt
7. 9.	vier Läufe hintereinander an der Kontingentgrenze gestorben	Rücksetzzeit merken, bis dahin aussetzen
10. 9.	der Auftrag über 128 KiB: „Argumentliste zu lang“	Auftrag über stdin, altes ins Archiv
25. 9.	zwei Läufe starten die Prüfung im Hintergrund, schreiben „ich warte“ und enden ohne Ergebnis	Verbot steht als erster Absatz im Auftrag
26. 9.	ein Ziel blockiert alle Läufe (B2)	ein einziger Lauf, dann Frage an den Nutzer
26. 9.	ein Lauf führt als offen, was schon dasteht	„Zuerst nachsehen, dann bauen“

Von gut 400 Läufen endeten 386 regulär, 18 mit Fehler, 3 an der Zeitschranke; 2 starteten nie (der zu lange Auftrag).

Was bleibt beim Nutzer

- ▶ **Die mathematische Richtigkeit der Aussagen.** `check_master.py` und `#print axioms` prüfen die Beweise, nicht, ob die Lean-Aussage das Manuskript trifft.
- ▶ **Die Richtung.** Läufe arbeiten ab, was dasteht, und sie sind gut darin. Was als Nächstes wichtig ist, sagen sie nicht.
- ▶ **Die Außenwirkung.** Mathlib und BrownianMotion bekommen nur, was ich freigebe.
- ▶ **Die Lesbarkeit.** Die Läufe schreiben viel. Das Inventar hat mehr als 65 000 Zeilen, und die READMEs redigiere ich selbst.